Name: K.Vaishali
Roll No: 2520030117
Sec: 8

PRACTICAL – 5

IPC USING ANONYMOUS PIPE & PIPELINE COMMAND EXECUTION

PART – A

Producer–Consumer Communication Using Anonymous Pipe

1. AIM

To implement a **producer-consumer communication system using an anonymous pipe**, where the **parent process acts as the producer** and the **child process acts as the consumer**, and to measure the communication efficiency.

2. OBJECTIVE

- To understand **Inter-Process Communication (IPC)**.
- To create an **anonymous pipe** between parent and child processes.
- To use the `fork()` system call to create a child process.
- To transfer data from the parent process to the child process.
- To measure the time taken for communication.

3. SYSTEM CALLS / FUNCTIONS USED

Function	Purpose
`pipe()`	Creates an anonymous pipe
`fork()`	Creates a child process
`write()`	Parent writes data into the pipe

`read()`	Child reads data from the pipe
`close()`	Closes the unused pipe ends
`wait()`	Parent waits for child
`gettimeofday()`	Measures communication time

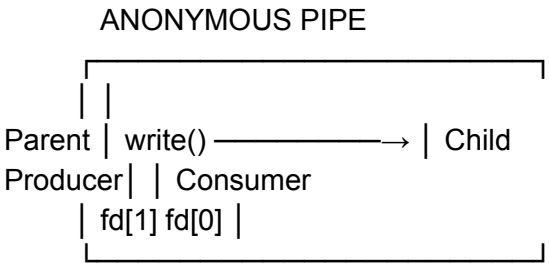
4. THEORY

An **anonymous pipe** is a mechanism used for communication between related processes.

A pipe has two ends:

fd[0] → Read end
fd[1] → Write end

In this program:



The **parent process produces the data** and writes it into the pipe.

The **child process consumes the data** by reading it from the pipe.

5. ALGORITHM

1. Start the program.
2. Create an anonymous pipe using `pipe()`.
3. Create a child process using `fork()`. 4. If the process is the parent:
    - Close the read end.
    - Generate data.
    - Write the data into the pipe.
    - Wait for the child process.

5. If the process is the child:
    - Close the write end.
    - Read data from the pipe.
    - Display the consumed data.
6. Calculate the communication time.
7. Display the result.
8. Stop.

6. PROGRAM

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <stdlib.h>

int main()
{
    int fd[2];
    pid_t pid;

    int data[5] = {10, 20, 30, 40, 50};
    int received[5];

    struct timeval start, end;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    gettimeofday(&start, NULL);
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid > 0)
    {
        /* Parent Process - Producer */
```

```
    close(fd[0]);

    printf("===== PRODUCER =====\n");

    write(fd[1], data, sizeof(data));

    printf("Produced data: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", data[i]);
    }

    printf("\n");

    close(fd[1]);

    wait(NULL);

    gettimeofday(&end, NULL);

    double time_taken =
        (end.tv_sec - start.tv_sec) * 1000000.0 +
        (end.tv_usec - start.tv_usec);

    printf("Communication time: %.2f microseconds\n",
           time_taken);
}
else
{
    /* Child Process - Consumer */
    close(fd[1]);

    read(fd[0], received, sizeof(received));

    printf("===== CONSUMER =====\n");

    printf("Consumed data: ");

    for (int i = 0; i < 5; i++)
    {
        printf("%d ", received[i]);
    }

    printf("\n");

    close(fd[0]);
```

```
        exit(0);
    }

    return 0;
}
```

7. PROCEDURE IN UBUNTU

Step 1: Open Ubuntu Terminal

Open your Ubuntu terminal.

Step 2: Create the C file

nano producer_consumer.c

Step 3: Paste the program

Paste the complete code above.

Step 4: Save the file

Press:
Ctrl + O

Then:

Enter

Then:

Ctrl + X

Step 5: Compile the program

gcc producer_consumer.c -o producer_consumer

If there is no error, compilation is successful.

Step 6: Execute the program

./producer_consumer

8. SAMPLE OUTPUT

```
===== PRODUCER =====
Produced data: 10 20 30 40 50
===== CONSUMER =====
Consumed data: 10 20 30 40 50
Communication time: 120.00 microseconds
```

Note: The communication time may be different on your computer.

9. RESULT

Thus, the **producer-consumer communication system was successfully implemented using an anonymous pipe**, where the parent process produced data and the child process consumed it. The communication time was also measured successfully.

PART – B

Implementing `ls -l | grep ".c"` Using System Calls

1. AIM

To implement the equivalent of the shell command

ls -l | grep ".c"

using the system calls **`fork()`**, **`pipe()`**, **`dup2()`**, and **`exec()`**.

2. OBJECTIVE

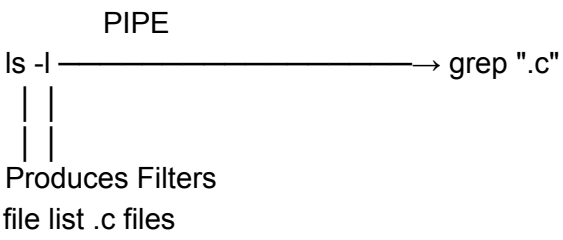
- To understand **IPC using pipes**.
- To create multiple child processes using `fork()`.
- To connect the output of one process to the input of another process.
- To understand how a shell implements the pipeline operator `|`.
- To execute Linux commands using `exec()`.

3. COMMAND TO BE IMPLEMENTED

Normally, in Linux we can execute:

ls -l | grep ".c"

The command works like this:



`ls -l` produces the list of files.

The output is sent through the pipe.

`grep ".c"` receives that output and displays only the lines containing

.c. 4. SYSTEM CALLS USED

System Call	Purpose
`pipe()`	Creates communication channel
`fork()`	Creates child processes
`dup2()`	Redirects input/output
`execlp()`	Executes Linux commands
`waitpid()`	Waits for child processes

`close()` Closes unused pipe ends

5. THEORY

The pipeline consists of **two processes**:

Process 1

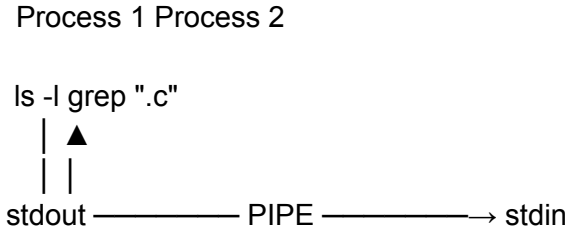
ls -l

This process generates the directory listing.

Process 2

grep ".c"
This process receives the output of `ls -l` and searches for

`.c`. The pipe connects them:



`dup2()` is used to connect:

ls stdout → pipe
grep stdin ← pipe

6. ALGORITHM

1. Start the program.
2. Create a pipe using `pipe()`.
3. Create the first child using `fork()`.
4. In the first child:
    - Close the read end of the pipe.
    - Redirect standard output to the pipe using `dup2()`.
    - Execute `ls -l` using `execlp()`.
5. Create the second child using `fork()`.
6. In the second child:
    - Close the write end of the pipe.

    - Redirect standard input from the pipe using `dup2()`.
    - Execute `grep ".c"` using `execlp()`.
7. Parent closes both pipe ends.
8. Parent waits for both child processes.
9. Stop the program.

7. PROGRAM

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int fd[2];
    pid_t pid1, pid2;

    if (pipe(fd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* First Child - ls -l */

    pid1 = fork();

    if (pid1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid1 == 0)
    {
        close(fd[0]);

        dup2(fd[1], STDOUT_FILENO);

        close(fd[1]);

        execlp("ls", "ls", "-l", NULL);
```

```
        perror("execlp ls");
        exit(1);
    }

    /* Second Child - grep ".c" */

    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid2 == 0)
    {
        close(fd[1]);

        dup2(fd[0], STDIN_FILENO);

        close(fd[0]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

    /* Parent Process */

    close(fd[0]);
    close(fd[1]);

    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    return 0;
}
```

8. PROCEDURE IN UBUNTU

Step 1: Create the file

nano ls_grep.c

Step 2: Paste the code

Paste the complete program above.

Step 3: Save

Ctrl + O
Enter
Ctrl + X

Step 4: Compile

gcc ls_grep.c -o ls_grep

Step 5: Run

./ls_grep

9. SAMPLE OUTPUT

Suppose your directory contains:

producer_consumer.c
ls_grep.c
test.c

The output will look similar to:

-rw-r--r-- 1 kruthi kruthi 1450 Sep 24 10:20 producer_consumer.c
-rw-r--r-- 1 kruthi kruthi 1200 Sep 24 10:25 ls_grep.c
-rw-r--r-- 1 kruthi kruthi 500 Sep 24 10:30 test.c

Only lines containing `.c` are displayed.

10. RESULT

Thus, the equivalent of the shell command `ls -l | grep ".c"` was successfully implemented using `fork()`, `pipe()`, `dup2()`, and `exec()` system calls.

```
 Birth: 2026-10-30 14:36:45.457805602 +0000
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
12213 hardlink.txt  12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ln original.txt hardlink.txt
ln: failed to create hard link 'hardlink.txt': File exists
vaishali@LAPTOP-QQD0F0TI:~$ ls -i original.txt hardlink.txt
12213 hardlink.txt  12213 original.txt
vaishali@LAPTOP-QQD0F0TI:~$
```

```
vaishali@LAPTOP-QQD0F0TI:~$ cd /mnt/c/Users/Vaishali/OneDrive/Desktop/OS
-bash: cd: /mnt/c/Users/Vaishali/OneDrive/Desktop/OS/Practical/Practical
vaishali@LAPTOP-QQD0F0TI:~$ echo "Hello Linux" > original.txt
vaishali@LAPTOP-QQD0F0TI:~$ ls -l original.txt
-rw-r--r-- 1 vaishali vaishali 12 Oct 30  2026 original.txt
vaishali@LAPTOP-QQD0F0TI:~$ stat original.txt
  File: original.txt
  Size: 12              Blocks: 8          IO Block: 4096   regular file
Device: 8,48    Inode: 12213       Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/vaishali)   Gid: ( 1000/vaishali)
Access: 2026-10-30 14:36:45.457805602 +0000
Modify: 2026-10-30 14:36:45.457805602 +0000
Change: 2026-10-30 14:36:45.457805602 +0000
 Birth: 2026-10-30 14:36:45.457805602 +0000
```